

Artisan Escrow Platform

Product Concept & Technical-Product Documentation

Hackathon: Monad Blitz Abuja

Location: Abuja, Nigeria

Target Market: Nigerian local service/artisan economy

Blockchain Infrastructure: Monad

Current Working Name: Artisan Escrow Platform

Product Stage: Hackathon MVP / Proof of Concept

1. Executive Summary

The Artisan Escrow Platform is a blockchain-powered marketplace designed to connect customers with local artisans while reducing the trust and payment problems that commonly occur during service transactions.

The platform is designed for everyday services such as plumbing, electrical work, carpentry, furniture making, painting, tiling, welding, appliance repair, and other skilled services.

The central idea is simple:

A customer posts a job, an artisan accepts it, the agreed payment is secured in escrow, the artisan completes the work, and the payment is released only after the customer confirms that the agreed work has been completed.

The blockchain operates primarily as the **trust and settlement infrastructure**, rather than something that users must understand before they can use the platform.

This distinction is important because the target audience includes artisans who may have little or no knowledge of cryptocurrency or blockchain technology.

The platform therefore aims to bridge the gap between **Web2-style everyday service transactions and Web3 financial infrastructure**.

2. The Problem

2.1 Trust Between Customers and Artisans

Local service transactions are frequently based on informal arrangements.

A customer may need an electrician, plumber, carpenter, or another skilled worker but may not know:

- Whether the artisan is reliable
- Whether they will actually show up
- Whether they will complete the work
- Whether the quality will meet expectations
- Whether the artisan will demand additional payment
- Whether payment should happen before or after the job

The artisan faces a similar problem.

They may worry that:

- The customer will refuse to pay
- Payment will be delayed
- The customer will change the agreed terms
- They will complete the work and have difficulty collecting their money

This creates a fundamental problem:

Both parties need to trust each other before the transaction can happen, but neither party necessarily has enough information to provide that trust.

3. The Existing Transaction Problem

Consider a simple example.

A customer's water pipe breaks.

They need a plumber immediately.

The customer finds a plumber through a referral, WhatsApp, social media, or another informal channel.

The conversation may become:

“How much will you charge?”

“N25,000.”

“Can you come now?”

“Yes.”

The customer may then wonder:

“Should I pay before the work?”

The artisan may think:

“Should I start working before receiving payment?”

This creates uncertainty for both sides.

If the customer pays first and the artisan does not complete the work, the customer bears the risk.

If the artisan works first and the customer refuses to pay, the artisan bears the risk.

The platform is designed to reduce this problem by introducing **escrow**.

4. Target Users

4.1 Primary User — Customers

The primary customer is an individual who needs a skilled worker to complete a specific service.

Examples include someone who needs:

- A leaking pipe repaired
- An electrical fault fixed
- Furniture constructed
- A room painted
- Tiles installed
- Welding work completed
- An air conditioner repaired
- A damaged appliance repaired
- Masonry or construction work completed

The initial geographic focus can be **Abuja**, allowing the team to validate the concept locally before considering expansion.

5. Primary User — Artisans

The second major user group consists of independent skilled workers.

Examples:

- Plumbers
- Electricians
- Carpenters
- Furniture makers
- Painters
- Tilers
- Bricklayers
- Welders
- Mechanics
- AC technicians
- Appliance repair technicians
- Other skilled service providers

The platform should not require these users to understand blockchain technology.

Their experience should be similar to using a normal service marketplace.

6. Secondary Users

As the platform develops, other users could include:

Businesses

Companies that regularly hire artisans.

Property Managers

People managing apartments, offices, hotels, or other properties.

Construction Businesses

Companies that need multiple skilled workers.

Organizations

Businesses or organizations requiring recurring maintenance services.

These users could eventually become higher-value customers because they may generate multiple transactions.

7. Core User Pain Points

Customer Pain Points

1. Difficulty finding reliable artisans.
2. Lack of trustworthy service history.
3. Fear of paying before work is completed.
4. Fear of poor-quality work.
5. Informal agreements.
6. Disagreements about what was actually promised.
7. Difficulty recovering money after a failed transaction.
8. Lack of structured dispute resolution.

Artisan Pain Points

1. Difficulty finding customers.
 2. Customers delaying payment.
 3. Customers refusing to pay after work is completed.
 4. Unclear job requirements.
 5. Price disputes.
 6. Lack of verifiable reputation.
 7. Dependence on informal referrals.
 8. Difficulty demonstrating previous successful work.
-

8. Proposed Solution

The platform combines four major components:

1. Marketplace

Customers can post jobs and artisans can discover and accept relevant jobs.

2. Escrow

The customer's payment is secured before the artisan begins work.

3. Verification & Reputation

Completed jobs can contribute to an artisan's reputation and transaction history.

4. Blockchain Infrastructure

Monad smart contracts can manage the escrow and payment-release process.

Together:

Marketplace + Escrow + Reputation + Blockchain

creates a trusted environment for local service transactions.

9. Core Value Proposition

For Customers

Find an artisan, secure your payment, and only release the funds when the agreed work is completed.

For Artisans

Find customers and know that the agreed payment has already been secured before you complete the job.

For the Web3 Ecosystem

Bring blockchain-powered financial trust to everyday users without requiring them to understand blockchain technology.

10. Core Product Flow

The basic transaction follows this sequence:

Customer needs a service

↓

Customer creates a job



Artisans view the job



Artisan accepts



Customer funds escrow



Payment becomes locked



Artisan performs the work



Artisan submits completion evidence



Customer reviews the work



If satisfied:

Customer approves



Escrow releases payment



Artisan receives payment



Transaction is recorded

If dissatisfied:

Customer raises dispute

↓

Funds remain locked

↓

Evidence is reviewed

↓

Resolution is made

↓

Funds are released according to the resolution

11. Example User Scenario

Scenario: Emergency Plumbing Repair

A customer discovers that a water pipe in their house has broken.

They open the platform and create:

Job

Title: Emergency Kitchen Pipe Repair

Description:

Kitchen water pipe is leaking heavily and needs to be repaired.

Budget: ₦25,000 equivalent

Location: Abuja

Deadline: Same day

Expected Outcome:

Leak repaired and water flow restored.

Step 1 — Artisan Discovery

Available plumbers can view the job.

An artisan can see:

- Job description
- Budget
- Location
- Deadline
- Customer requirements

The artisan accepts the task.

Step 2 — Payment Security

The customer funds the agreed amount through the platform.

The platform places the funds into the escrow mechanism.

The interface shows:

Payment Secured

The artisan knows that the customer's payment has already been committed to the job.

Step 3 — Work

The artisan visits the customer and performs the repair.

Step 4 — Completion

The artisan marks the job as completed.

They can submit evidence such as:

- Before photo
 - After photo
 - Completion note
 - Optional video
-

Step 5 — Customer Review

The customer reviews the work.

They have two primary choices:

Approve Work

The customer confirms:

Work completed satisfactorily.

The escrow releases the payment.

Raise Dispute

The customer indicates that the agreed work was not satisfactorily completed.

The funds remain locked while the dispute is reviewed.

12. Escrow Mechanism

Escrow is the central financial mechanism of the platform.

Instead of sending money directly to the artisan immediately, the customer deposits the agreed amount into an escrow smart contract.

The smart contract holds the funds according to predefined conditions.

Escrow States

1. Job Created

The customer has created the job.

No payment has been secured yet.

2. Job Funded

The customer deposits the agreed amount.

The escrow is now active.

3. Job Accepted

An artisan accepts the job.

4. Work in Progress

The artisan is completing the task.

5. Work Submitted

The artisan indicates that the job is complete.

6. Customer Approval

The customer confirms completion.

7. Payment Released

The smart contract releases the funds.

13. Why Escrow Matters

The escrow mechanism creates a balanced incentive.

Customer Protection

The customer does not simply hand money directly to an unknown artisan.

Artisan Protection

The artisan does not have to rely on a verbal promise that the customer will pay later.

Platform Protection

The platform creates a structured transaction history instead of relying on informal conversations.

The key principle is:

The money is committed before the work begins but released after the agreed conditions are satisfied.

14. Non-Crypto User Experience

This is one of the most important aspects of the product.

The target artisan should not need to understand:

- Blockchain
- Smart contracts
- Gas
- Private keys
- Network confirmations
- Token standards
- Blockchain explorers

The product should hide as much technical complexity as possible.

Instead of showing:

“0.00X MON deposited into contract 0x...”

the artisan should see:

Job Payment Secured
¥25,000 equivalent
Status: Work in Progress

The technology should operate underneath the user experience.

15. Proposed Payment Experience

There are two possible stages of the product.

Hackathon MVP

The simplest implementation is to demonstrate the actual blockchain escrow using a supported digital asset/test environment.

This allows the team to prove that:

Funds → Smart Contract → Approval → Release

actually works.

The artisan-facing interface can still present the transaction in simple language.

Future Production Version

The long-term product should abstract cryptocurrency away from artisans as much as possible.

An artisan should ideally be able to receive the value of their completed work without having to become a crypto expert.

This may eventually require integration with appropriate payment providers, wallets, stablecoins, or fiat settlement/off-ramp infrastructure.

This is intentionally treated as a **future product layer**, rather than something the team must solve completely during the hackathon.

16. Dispute Resolution

A major weakness of simple escrow systems is that:

A smart contract can verify whether a transaction happened, but it cannot automatically determine whether a plumber actually fixed a pipe correctly.

Therefore, the product needs a human-assisted dispute mechanism.

17. Proposed MVP Dispute Process

If the customer is dissatisfied:

Step 1

Customer selects:

Raise Dispute

The payment remains locked.

Step 2

Both parties can provide evidence.

Evidence could include:

- Job description
- Agreed price
- Photos
- Videos
- Completion evidence
- Messages
- Other relevant documentation

Step 3

A designated platform reviewer/admin examines the evidence.

Step 4

The reviewer can recommend one of three outcomes:

Full payment to artisan

if the work was properly completed.

Full refund to customer

if the agreed work was not completed.

Partial settlement

if there is evidence that part of the work was completed or the situation warrants a split.

Step 5

The appropriate transaction is executed.

18. Future Dispute Resolution

As the platform grows, the centralized reviewer model could evolve.

Possible future systems include:

- Multiple independent arbitrators
- Reputation-weighted dispute resolution
- Community arbitration
- Verified service inspectors
- Third-party dispute providers
- Decentralized arbitration mechanisms

However, these should **not** be part of the first MVP unless the technical team has sufficient time.

19. Preventing Disputes Before They Happen

The platform should not rely only on dispute resolution.

It should reduce ambiguity at the beginning of every transaction.

Every job should contain:

Job Description

What exactly needs to be done?

Budget

How much will the customer pay?

Deadline

When should it be completed?

Expected Outcome

What qualifies as successful completion?

Materials

Who provides the materials?

Evidence

What should be submitted when the job is complete?

This turns:

“Fix my pipe.”

into:

“Repair kitchen pipe leak, restore water flow, complete by 6 PM, materials included, provide completion photo.”

The clearer the agreement, the easier it becomes to determine whether the work was completed.

20. Artisan Reputation

A major future feature is a structured reputation system.

After every completed job, the platform can record:

- Job completed
- Customer rating
- Completion history
- Dispute history
- Successful payment history
- Number of jobs
- Service category

Example:

Musa — Plumber

Jobs completed: 42

Successful transactions: 40

Average rating: 4.8/5

Disputes: 1

Completed on time: 38/42

This gives customers more information than simply asking:

“Do you know a good plumber?”

21. Verification

The platform can eventually introduce artisan verification.

Potential verification levels:

Basic

Phone/email verification.

Verified

Identity verification.

Skilled

Evidence of professional experience, certifications, references, or other appropriate credentials.

Trusted

A strong history of successfully completed platform transactions.

Verification should be designed carefully so that the platform does not unnecessarily exclude legitimate informal artisans.

22. MVP Features

Because the hackathon build time is limited, the MVP should focus on proving the core transaction.

Customer

- Create a job
- Add description
- Set budget
- Set deadline
- View available artisans
- Select artisan
- Fund escrow
- View job status
- Approve completed work
- Raise dispute

Artisan

- Create profile
- Add skill/category
- View available jobs
- Accept a job
- View job details
- Mark job as complete
- Submit completion evidence
- View payment status

Escrow

- Create escrow
- Fund escrow
- Lock funds
- Release funds
- Maintain dispute/locked state

Basic Reputation

- Completed jobs
- Rating
- Transaction history

23. Recommended 8-Hour Hackathon Scope

The team should not attempt to build the entire long-term product.

The MVP should prove **one complete transaction**.

Demo Scenario

Customer

Posts:

Emergency Plumbing Repair — ₦25,000



Artisan

Accepts the job.



Customer

Funds escrow.



System

Displays:

Payment Secured.



Artisan

Completes the job.



Artisan

Uploads completion proof.



Customer

Approves.

↓

Smart Contract

Releases payment.

↓

System

Records the successful transaction.

24. Optional Demo — Dispute

If the developer has enough time, demonstrate a second path:

Customer rejects the work.

↓

Funds remain locked.

↓

Customer submits dispute.

↓

Artisan submits evidence.

↓

Admin/reviewer resolves.

↓

Funds are released according to the resolution.

This demonstrates that the product isn't simply:

“Payment → release.”

It demonstrates an actual **trust infrastructure**.

25. What Should NOT Be Required for the MVP

The team should avoid scope creep.

The following should be considered future features unless the developer has significant time:

- Full fiat settlement
- Complex identity verification
- Advanced AI matching
- Full decentralized arbitration
- Mobile applications
- Advanced reputation protocol
- Multi-city operations
- Large-scale artisan onboarding
- Complex recommendation systems
- Multiple blockchain integrations
- Advanced tokenomics
- Full business-account infrastructure

The objective is:

One working transaction is better than twenty unfinished features.

26. Monad Integration

Monad serves as the blockchain infrastructure supporting the escrow system.

The team can use smart contracts to handle:

- Escrow deposits
- Locked funds
- Conditional payment release
- Transaction records
- Potential future reputation data

Monad's EVM compatibility allows the team to use familiar Ethereum development tooling and Solidity-based smart contracts.

Its high-performance architecture and low transaction costs are relevant to a platform that could eventually handle many relatively small service transactions.

However, the product should not claim that Monad itself solves the trust problem.

The **smart contract handles the financial conditions**.

The **platform handles the marketplace and human verification**.

The **dispute system handles disagreements**.

Together, these components create the trust layer.

27. Why Blockchain Is Necessary

The product should have a clear reason for using blockchain.

The blockchain provides:

Programmable Escrow

Funds can be locked according to predefined rules.

Transparent Transactions

Transactions can be independently verified on-chain.

Conditional Settlement

Payment can be released when the agreed transaction condition is met.

Reduced Dependence on Manual Payment Handling

The platform does not need to manually execute every successful payment.

Portable Transaction History

Future versions can use on-chain transaction history as part of reputation.

The key question should always be:

What does blockchain improve here?

The answer is:

It makes the payment commitment programmable and verifiable rather than relying entirely on one party's promise.

28. Product Architecture

At a high level, the platform consists of several layers.

Layer 1 — User Interface

Customer and artisan dashboards.

Layer 2 — Application Backend

Handles:

- User accounts
- Job listings
- Profiles
- Job status
- Evidence
- Notifications
- Platform administration

Layer 3 — Smart Contract

Handles:

- Escrow
- Fund locking
- Payment release
- Relevant transaction states

Layer 4 — Blockchain

Monad provides the underlying transaction infrastructure.

29. Customer Dashboard

The customer dashboard could contain:

Active Jobs

- Job title
- Artisan
- Amount secured
- Status
- Deadline

Completed Jobs

- Completed work
- Amount paid
- Artisan
- Rating

Disputes

- Active dispute
 - Evidence
 - Resolution status
-

30. Artisan Dashboard

The artisan dashboard could contain:

Available Jobs

Jobs matching the artisan's service category.

Active Job

- Customer
- Location
- Description
- Payment secured
- Deadline

Completed Jobs

Previous work history.

Reputation

- Rating
- Completed jobs
- Successful transactions

Payment Status

Secured

Pending approval

Released

31. Marketplace Matching

The initial MVP can use basic categories.

For example:

Customer searches:

Plumbing

The platform shows:

- Available plumbers
- Location
- Rating
- Completed jobs

More advanced matching can come later.

Potential future criteria:

- Distance
- Availability
- Price
- Rating
- Experience
- Previous job category

- Urgency
-

32. Business Model

The primary business model can be a small transaction fee.

Example:

A customer funds a:

₦25,000 job

The platform charges a small percentage for providing the marketplace and escrow service.

For example:

2% platform fee = ₦500

The actual percentage would need to be validated through market research and operating costs.

33. Additional Future Revenue

Possible revenue streams include:

Premium Artisan Profiles

Artisans pay for enhanced visibility.

Featured Listings

Artisans or businesses can promote their services.

Business Accounts

Businesses pay for advanced hiring and transaction management.

Subscription

Frequent users could receive premium features.

Verification

Optional paid verification services could be considered where appropriate.

The platform should avoid charging unnecessary fees that discourage low-income artisans.

34. Competitive Differentiation

The platform is not simply trying to be another directory of artisans.

Its key differentiation is:

The transaction itself is protected.

Traditional marketplace:

Find artisan → contact artisan → negotiate → pay → hope

Artisan Escrow:

Find artisan → define job → secure payment → complete work → verify → release

The product therefore combines:

Discovery + Agreement + Escrow + Verification + Reputation

35. Web2-to-Web3 Bridge

One of the strongest aspects of the concept is that users can benefit from blockchain without initially needing to understand blockchain.

A customer may simply want:

“I need a plumber.”

An artisan may simply want:

“I want to get paid for my work.”

Neither needs to begin with:

“I want to use blockchain.”

The platform introduces blockchain as infrastructure.

Over time, users can learn:

“My payment was protected by a smart contract.”

This creates a practical introduction to Web3 rather than asking users to adopt crypto first.

36. User Education

Education should be built into the interface without overwhelming users.

Examples:

Payment Secured

“Your payment has been safely locked until the agreed work is completed.”

Payment Released

“The customer approved the completed work. Your payment has been released.”

Dispute

“The payment is temporarily locked while both sides provide information about the job.”

This makes blockchain concepts understandable through real actions.

37. Security Considerations

Because the platform handles funds, security is critical.

The team should consider:

- Smart-contract security
- Access control
- Admin privileges
- Protection against unauthorized withdrawals

- Reentrancy protection
- Input validation
- Transaction confirmation
- Dispute manipulation
- Fake job listings
- Fake completion evidence
- Account takeover
- Fraudulent users

For the hackathon, the team should clearly communicate that the MVP is a **prototype** and not yet production-ready for significant real funds.

38. Trust & Safety Risks

Potential risks include:

Fake Artisans

Solution:

Verification and transaction reputation.

Fake Jobs

Solution:

Account controls and monitoring.

Customer Fraud

Example:

Customer falsely claims that work wasn't completed.

Solution:

Evidence-based dispute resolution.

Artisan Fraud

Example:

Artisan claims work was completed without actually completing it.

Solution:

Completion evidence and customer verification.

Payment Manipulation

Solution:

Smart-contract-controlled escrow.

39. Key Product Principle

The platform should never promise:

“Blockchain eliminates fraud.”

That would be inaccurate.

Instead:

“The platform reduces payment and trust risks by securing funds, creating clear transaction conditions, maintaining evidence, and providing a structured dispute process.”

This is a much more credible product claim.

40. Success Metrics

For an early MVP, success can be measured through:

Marketplace

- Number of jobs created
- Number of artisans registered
- Number of jobs accepted

Transactions

- Number of escrow transactions
- Total transaction value
- Successful payment releases

Trust

- Percentage of successfully completed jobs
- Dispute rate
- Average customer rating
- Artisan retention

User Experience

- Time required to create a job
- Time required to accept a job
- Time from completion to payment release

41. Future Roadmap

Phase 1 — Hackathon

Prove:

Post → Fund → Work → Approve → Release

Phase 2 — Local Pilot

Test with real artisans and customers in Abuja.

Focus on:

- Plumbing
- Electrical work
- Carpentry
- Repairs

Collect real user feedback.

Phase 3 — Trust Layer

Introduce:

- Artisan verification
- Reputation
- Ratings

- Transaction history
- Better dispute resolution

Phase 4 — Payments

Improve payment abstraction so non-crypto users can transact without understanding blockchain.

Phase 5 — Expansion

Expand beyond Abuja into other Nigerian cities.

Phase 6 — Broader Services

Expand into:

- Construction
- Maintenance
- Business services
- Property management
- Commercial contracts

42. Long-Term Vision

The long-term vision is not simply to create another artisan marketplace.

It is to build a **trusted transaction infrastructure for informal and semi-formal service work**.

The marketplace is the entry point.

Escrow is the trust mechanism.

Reputation becomes the long-term asset.

Blockchain provides the settlement infrastructure.

The ultimate vision is:

A place where customers can confidently hire skilled workers and artisans can confidently work knowing their agreed payment is secured.

43. The Core Product in One Sentence

A blockchain-powered artisan marketplace that protects both customers and skilled workers by securing payment in escrow until agreed work is completed and approved.

44. Short Pitch

We are building a blockchain-powered marketplace for local artisans where customers can post jobs, artisans can accept them, and the agreed payment is secured in escrow until the work is completed and approved. The platform hides the complexity of crypto, allowing everyday users to benefit from blockchain-powered payment protection without needing to understand Web3.

45. 30-Second Demo Explanation

“Imagine your water pipe breaks and you need a plumber. You post the job on our platform, an available plumber accepts it, and you secure the agreed payment in escrow. The plumber completes the repair and submits proof. Once you confirm that the work is satisfactory, the smart contract releases the payment. If there's a disagreement, the funds remain locked while the issue is reviewed. So instead of relying entirely on trust between strangers, we use escrow, evidence, reputation, and blockchain infrastructure to make the transaction safer.”

46. What the Team Needs to Decide Before Building

The following decisions should be confirmed by the team:

Product

- Final product name
- Initial artisan categories
- Initial geographic area
- Customer onboarding process
- Artisan onboarding process

Payment

- Exact token used for MVP
- Whether the customer or platform pays gas
- Whether users interact directly with wallets
- How payment amounts are displayed
- Future fiat/off-ramp strategy

Escrow

- Exact smart-contract states
- Conditions for release
- Cancellation rules
- Expiration rules

Disputes

- Who can open a dispute
- How long disputes remain open
- Who reviews disputes
- What evidence is required
- Whether partial refunds are possible

Technical

- Smart-contract architecture
- Frontend
- Backend
- Wallet integration
- Monad testnet/mainnet environment as applicable
- Deployment process

Hackathon

- Exact MVP scope
- Developer capacity
- Team responsibilities
- Demo scenario
- Pitch narrative

47. Recommended MVP Decision

If the developer's available time or technical capacity is limited, the team should prioritize the following order:

Priority 1: Job creation

Priority 2: Job acceptance

Priority 3: Escrow deposit

Priority 4: Work completion

Priority 5: Customer approval

Priority 6: Smart-contract payment release

Priority 7: Basic dispute state

Priority 8: Basic profiles/reputation

Everything else should be considered optional.

The most important thing is to demonstrate one complete, functioning transaction.

48. Final Product Philosophy

The platform should be built around one simple principle:

Make trust easier, not crypto more complicated.

The customer should think:

“My money is protected.”

The artisan should think:

“My payment is secured.”

The developer should know:

“The smart contract controls the escrow.”

And the judge should understand:

“This team took a real local problem and used blockchain where it actually adds value.”

That is the central product story.